

ACTIVE DIRECTORY CERTIFICATE SERVICES

AD CS → EFS

Internal PKI Certificate Enrollment & Encrypting File System Demonstration

Lab: healthcare.local | CA: HEALTHCARE-ROOT-CA | Platform: Windows Server 2022 + Windows 8

AD CS Internal Certificate Authority Role	EFS Encrypting File System	PKI Public Key Infrastructure Built	ISO 27001 Controls Demonstrated
--	--------------------------------------	--	--

Date: 24 August 2026

Table of Contents

1. Executive Summary	3
2. Project Objectives	4
3. Lab Environment & Architecture	5
4. Conceptual Foundation — PKI, AD CS, and EFS	6
4.1 What is a Public Key Infrastructure (PKI)?	6
4.2 What is Active Directory Certificate Services (AD CS)?	7
4.3 What is the Encrypting File System (EFS)?	7
4.4 How AD CS and EFS Work Together	8
5. Implementation — Step-by-Step	9
5.1 Step 1 — Install the AD CS Role (Server Manager)	9
5.2 Step 2 — Configure the Enterprise Root CA	11
5.3 Step 3 — Enable the EFS Certificate Template	13
5.4 Step 4 — Create Domain Users (ali.test & other.test)	14
5.5 Step 5 — Enroll ali.test for an EFS Certificate	15
5.6 Step 6 — Verify the Issued Certificate	16
5.7 Step 7 — Create the Confidential Test File	17
5.8 Step 8 — Encrypt the File with EFS	18
5.9 Step 9 — Verify Encryption with cipher Command	19
5.10 Step 10 — Test Unauthorised Access (other.test)	20
6. Technical Deep Dive — How EFS Encryption Works	21
7. Verification & Test Results	22
8. Security Concepts Demonstrated	23
9. Framework Alignment — ISO 27001 & NIST CSF	24
10. Key Lessons & Sysadmin Notes	25
11. Recommended Next Phases	26
12. Screenshot Evidence Index	27

1. Executive Summary

This document presents the complete technical implementation record for an Active Directory Certificate Services (AD CS) to Encrypting File System (EFS) project, conducted in an on-premises healthcare domain lab environment. The project establishes a functional internal Public Key Infrastructure (PKI) from scratch — deploying an Enterprise Root Certificate Authority, enrolling domain users for EFS-capable certificates, and demonstrating cryptographically enforced file confidentiality.

The lab environment simulates a healthcare organisation's IT infrastructure: a Windows Server 2022 domain controller running Active Directory Domain Services (AD DS), DNS, and AD CS under the domain healthcare.local. A Windows 8 client machine joined to the domain serves as the endpoint for certificate enrollment and EFS testing.

The project demonstrates a complete, end-to-end cryptographic trust chain: Active Directory establishes the user's identity; AD CS issues a trusted certificate binding that identity to a public/private key pair; EFS uses that certificate to encrypt a file so that only the certificate holder (ali.test) can decrypt it; and a second domain user (other.test) — despite having a valid domain account — cannot open the encrypted file because they do not possess ali.test's private key. This is a fundamental demonstration of the principle that cryptographic access control operates independently of, and provides stronger guarantees than, traditional NTFS permission-based access control.

2. Project Objectives

This project was designed to demonstrate a practical, end-to-end implementation of enterprise identity-linked cryptography using only built-in Windows Server and client components — requiring no third-party software.

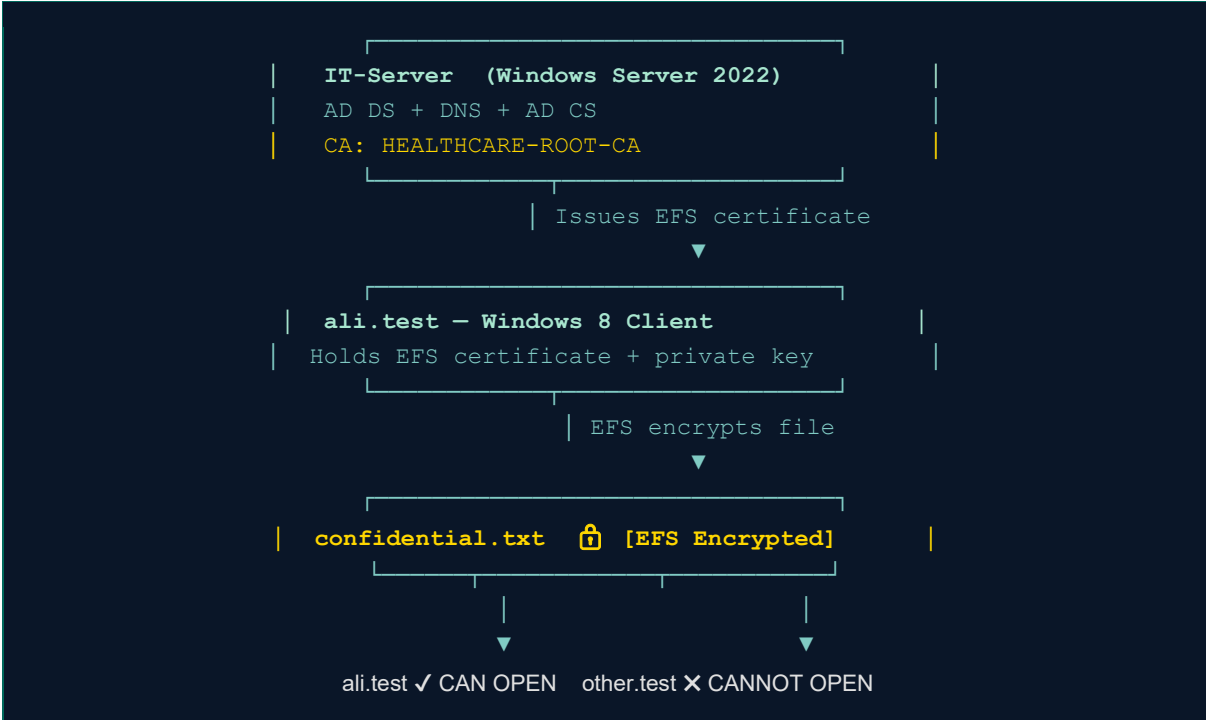
#	Objective	How the Lab Demonstrates It
O-1	Deploy an internal Enterprise Root Certificate Authority using AD CS	AD CS role installed on IT-Server (Windows Server 2022); Enterprise Root CA named HEALTHCARE-ROOT-CA configured with a new CA private key
O-2	Issue an EFS-capable certificate to a domain user	ali.test domain user enrolls for a certificate through the Active Directory Enrollment Policy using the Certificates MMC snap-in; certificate issued by HEALTHCARE-ROOT-CA
O-3	Use the user certificate to encrypt a file with EFS	ali.test encrypts C:\EFS-Test\confidential.txt using File Properties → Advanced → Encrypt contents — EFS uses the enrolled certificate's public key to protect the file-encryption key
O-4	Demonstrate that another user cannot decrypt the file	other.test (valid domain account, no EFS certificate for this file) attempts to open the encrypted file and receives an access denial — proving cryptographic access control
O-5	Show the relationship between AD identity, PKI, certificates, encryption, and access control	The complete chain is traced: AD DS (identity) → AD CS (certificate issuance) → Certificate (public/private key pair) → EFS (file encryption using public key) → Private key (decryption capability)

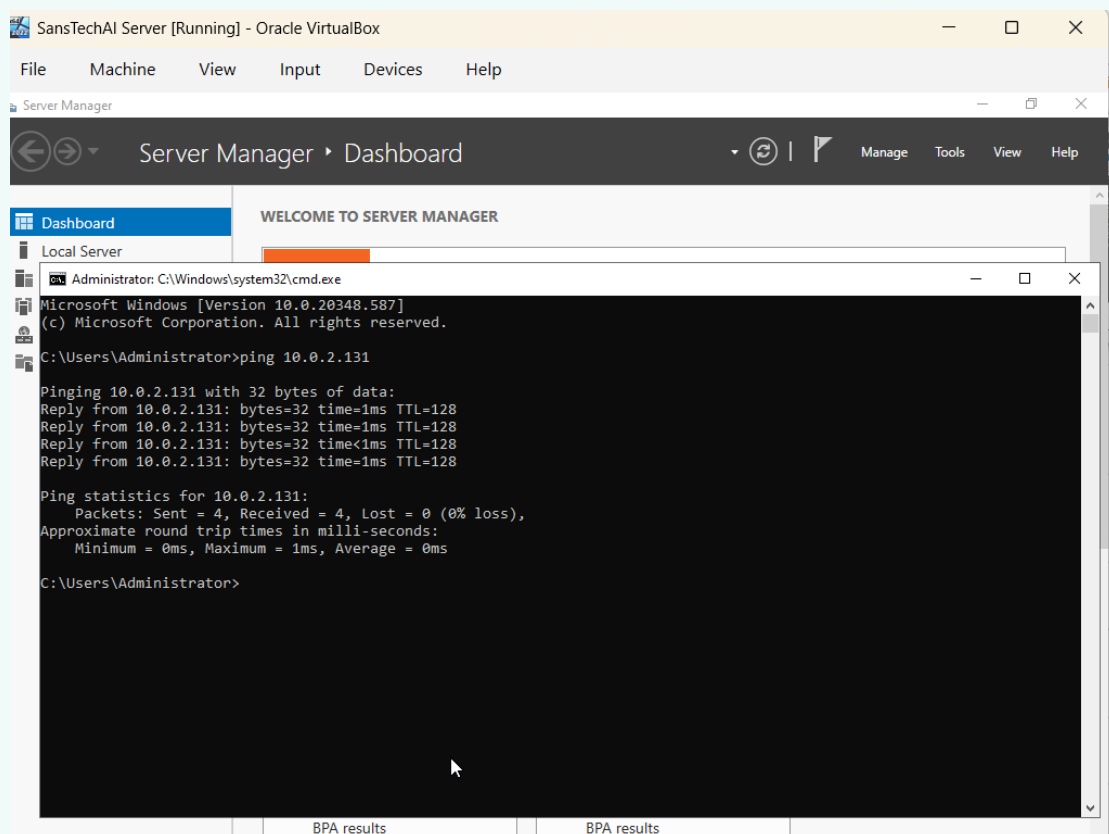
3. Lab Environment & Architecture

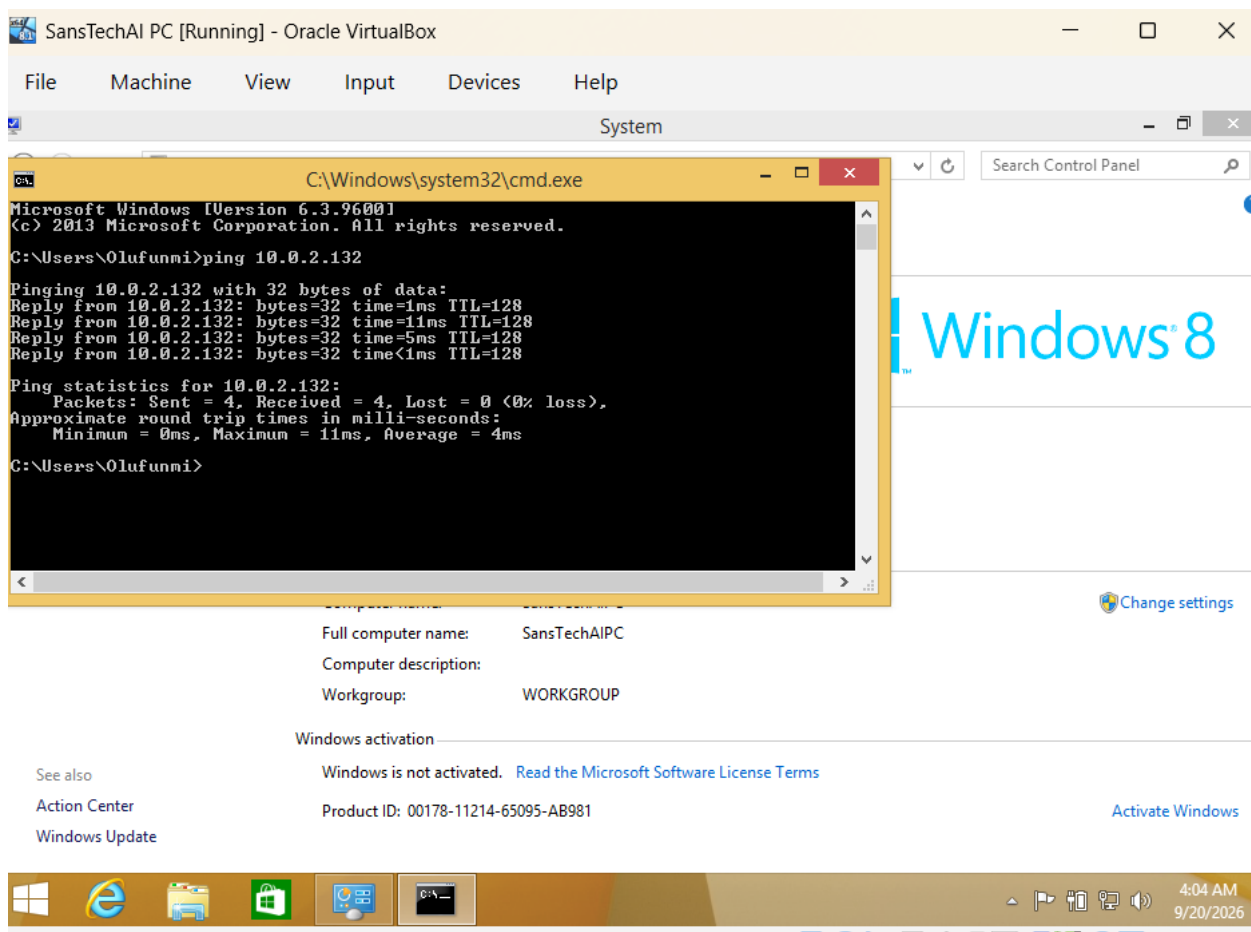
3.1 Lab Components

Component	Platform / Role	Name / Value	Purpose
Domain Controller	Windows Server 2022	IT-Server	Hosts AD DS, DNS, and the AD CS Certificate Authority role
Active Directory Domain	healthcare.local	healthcare.local	Central identity and trust environment for the lab
Certificate Authority	AD CS Enterprise Root CA	HEALTHCARE-ROOT-CA	Issues and signs internal certificates; root of trust for the PKI
Client Workstation	Windows 8 (domain-joined)	WIN8-CLIENT	Certificate enrollment endpoint; EFS test machine
Test User 1 (Owner)	Domain User Account	ali.test	Enrolls for EFS certificate; creates and encrypts the test file
Test User 2 (Attacker)	Domain User Account	other.test	Valid domain account — attempts unauthorised decryption of ali.test's encrypted file

3.2 Architecture Diagram



 [SCREENSHOT]



4. Conceptual Foundation — PKI, AD CS, and EFS

4.1 What is a Public Key Infrastructure (PKI)?

A Public Key Infrastructure (PKI) is the framework of policies, procedures, hardware, software, and people needed to create, manage, distribute, use, store, and revoke digital certificates. At its core, PKI solves a fundamental problem in cryptography: how do you know that a public key actually belongs to the person it claims to belong to?

PKI addresses this by introducing a trusted third party — the Certificate Authority (CA) — that digitally signs certificates. A certificate is a document binding a public key to an identity (e.g. ali.test@healthcare.local). Because the CA's signature can be verified by anyone who trusts the CA, a certificate provides proof of identity with cryptographic strength.

PKI Key Concepts

Certificate Authority (CA): The trusted entity that issues and signs certificates. In this lab: HEALTHCARE-ROOT-CA

Certificate: A digitally signed document containing a public key, the identity it belongs to, its validity period, and the CA's signature

Public Key: The part of the key pair that can be freely shared — used to encrypt data or verify signatures

Private Key: The secret part of the key pair — held only by the certificate owner and required to decrypt data encrypted with the corresponding public key

Trust Chain: A path of cryptographic trust from a root CA, through any intermediate CAs, down to the end-entity certificate. Trust in the root CA means trust in everything it signs

4.2 What is Active Directory Certificate Services (AD CS)?

Active Directory Certificate Services (AD CS) is a Windows Server role that provides the services needed to create and manage a PKI tied to your Active Directory environment. An Enterprise CA installed as part of AD CS is tightly integrated with AD DS — it reads domain user accounts and groups from Active Directory to determine who is authorised to enroll for which certificate templates, and it publishes issued certificates back to Active Directory.

AD CS Capability	What It Means in This Lab
Enterprise Root CA	HEALTHCARE-ROOT-CA is the top-level trust anchor — it signs all certificates issued in this domain
Certificate Templates	Pre-configured certificate profiles (e.g. the EFS template) define what a certificate can be used for and who can enroll for it
Active Directory Integration	The CA uses AD to verify ali.test is a valid domain user before issuing a certificate — AD identity is the basis of certificate trust
Automatic Enrollment	Through Group Policy, AD CS can automatically provision certificates to all users or computers meeting a template's criteria (demonstrated in the next phase)
Certificate Revocation	The CA can revoke a certificate before its expiry if a user's private key is compromised — immediately invalidating their ability to decrypt files

4.3 What is the Encrypting File System (EFS)?

The Encrypting File System (EFS) is a feature of the Windows NTFS file system that provides file-level encryption built directly into the operating system. When a file is encrypted with EFS, Windows transparently encrypts and decrypts it as the authorised user reads and writes it — but any other user who attempts to access the file receives an 'Access is denied' error, even if they have NTFS read permissions.

Crucially, EFS is not the same as NTFS permissions. NTFS permissions control who can see a file in the directory listing and access its metadata — they are enforced by the Windows security subsystem. EFS encryption protects the actual content of the file using cryptography — it is enforced by the mathematics of asymmetric encryption and cannot be bypassed by administrators with NTFS access unless they also hold the correct decryption key.

EFS vs NTFS Permissions — A Critical Distinction
NTFS Permissions (traditional access control):

- Controlled by ACLs (Access Control Lists) in the file system
- Enforced by Windows security subsystem (kernel)
- Can be overridden by domain admins with 'Take Ownership' right
- Protects access at the file-system level

EFS Encryption (cryptographic access control):

- Controlled by possession of the private key
- Enforced by asymmetric cryptography (mathematics, not policy)
- Cannot be bypassed by admins without the private key (unless a DRA is configured)
- Protects the actual file contents — even if the file is copied to another system

4.4 How AD CS and EFS Work Together

→	Component	Role in the Chain
1	Active Directory (AD DS)	Provides the user identity — ali.test is a domain account in healthcare.local. AD is the identity foundation.
2	AD CS / HEALTHCARE-ROOT-CA	Issues an EFS certificate to ali.test. The certificate contains ali.test's public key and is signed by the CA, establishing trust.
3	Certificate	The issued certificate binds ali.test's public key to their AD identity. The corresponding private key is stored securely in ali.test's user profile on the client.
4	EFS Encryption	When ali.test encrypts a file, EFS generates a random symmetric File Encryption Key (FEK). EFS then encrypts the FEK using ali.test's public key (from the certificate) and stores the encrypted FEK in the file's metadata.
5	EFS Decryption (ali.test)	When ali.test opens the file, EFS uses ali.test's private key to decrypt the FEK, then uses the FEK to decrypt the file content. This happens transparently.
6	Denial (other.test)	When other.test attempts to open the file, EFS finds the encrypted FEK but other.test's private key cannot decrypt it — because it was encrypted for ali.test's public key only.

5. Implementation — Step-by-Step

The following ten steps document the complete implementation in the order it was performed. Each step includes the exact navigation path, configuration values used, and screenshot evidence placeholders at every key point. The document follows the authoritative sysadmin convention: record every decision and its rationale, not just the actions.

01

Install the AD CS Role (Server Manager)

Windows Server 2022 — Server Manager

5.1 Step 1 — Install the AD CS Role

The Active Directory Certificate Services role was installed on IT-Server using Server Manager's Add Roles and Features Wizard. The Certification Authority role service was selected — this is the core component that will act as HEALTHCARE-ROOT-CA. No other AD CS role services (NDES, CES, CEP, Web Enrollment) were installed for this foundational lab.

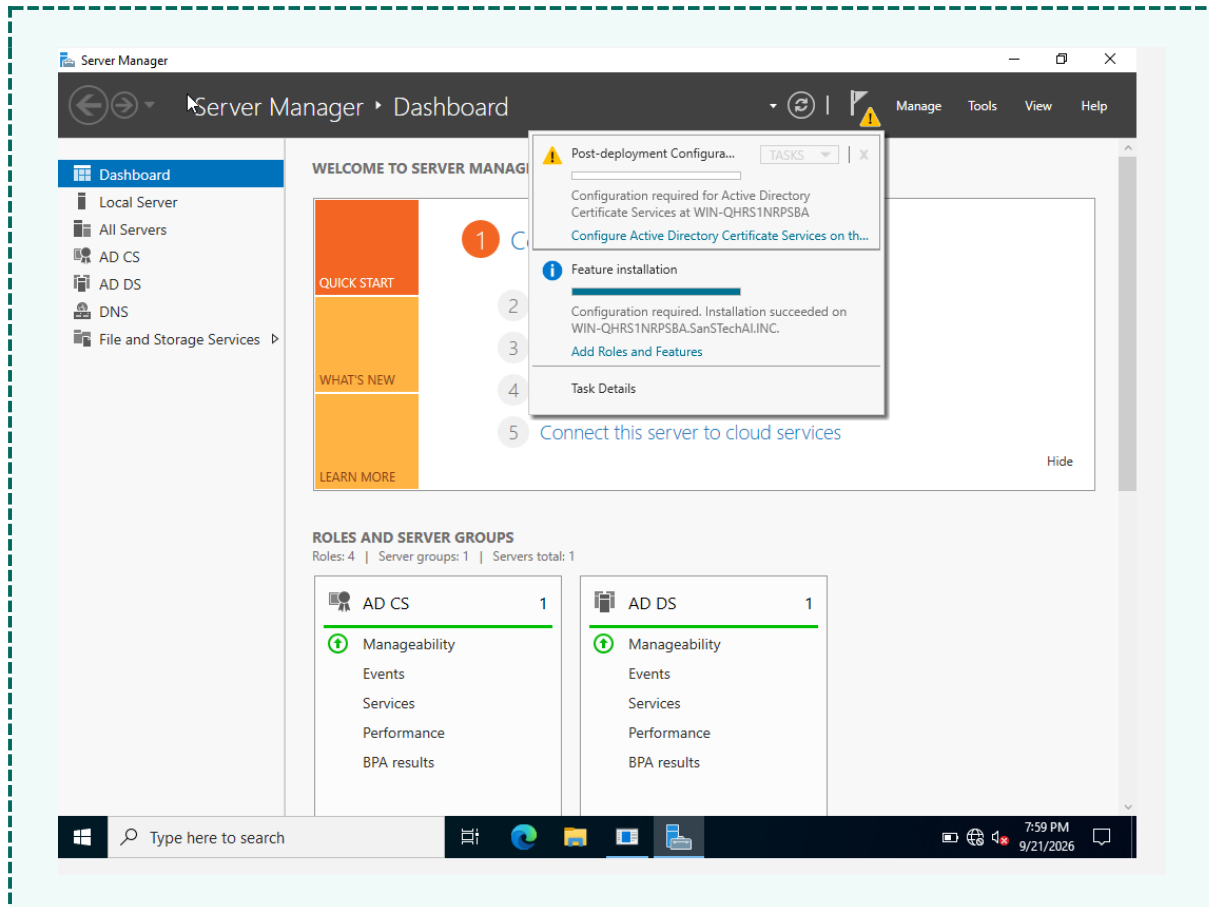
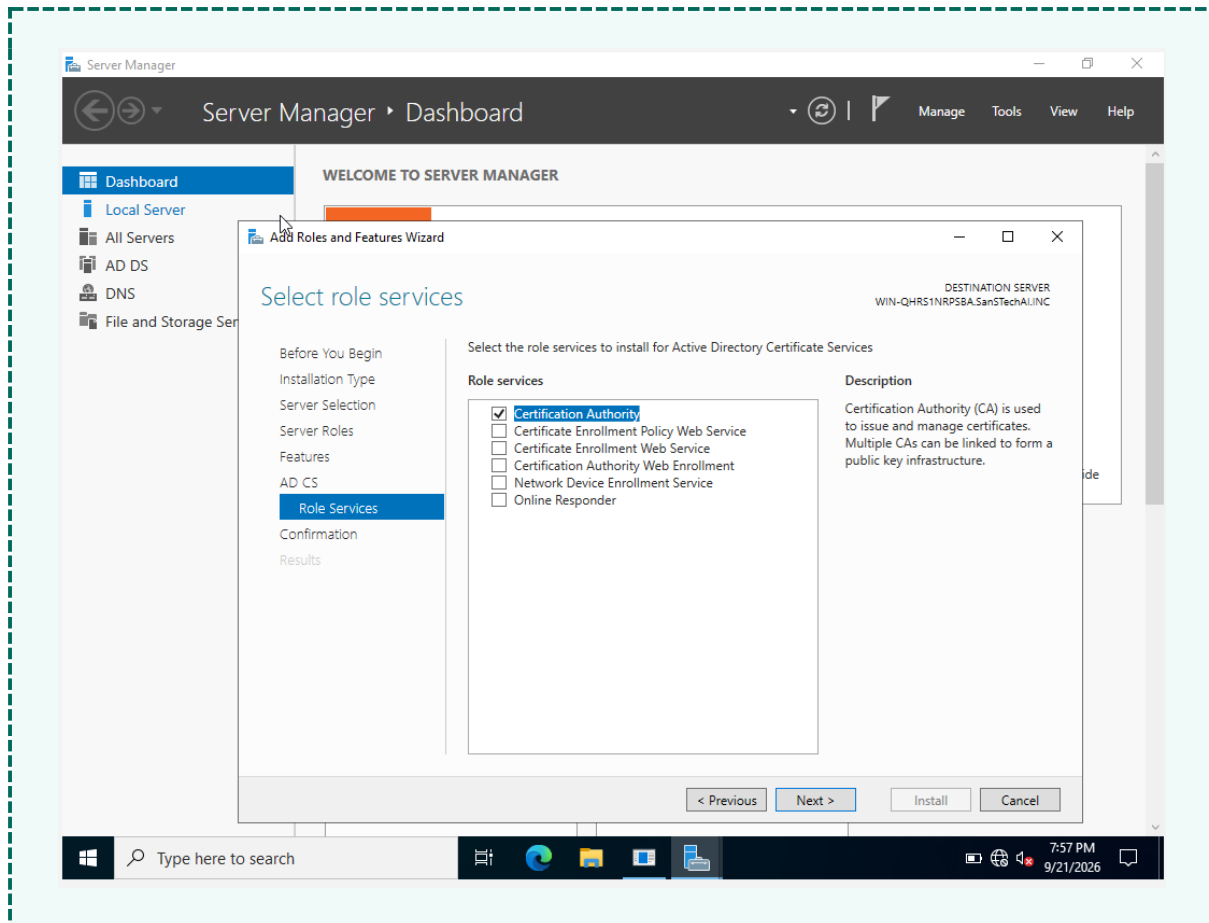
Navigation Path

- Server Manager > Dashboard > Add roles and features
- Installation Type: Role-based or feature-based installation
- Server Selection: IT-Server (the local server)
- Server Roles: Active Directory Certificate Services — checked
- Role Services: Certification Authority — checked (only this one)
- Confirmation page: checked Restart the destination server automatically if required
- Clicked Install — waited for installation to complete

► PowerShell — AD CS Role Installation (Equivalent Command)

```
# PowerShell equivalent — install AD CS role
Install-WindowsFeature -Name AD-Certificate -IncludeManagementTools

# Verify the role was installed
Get-WindowsFeature -Name AD-Certificate
[X] Active Directory Certificate Services    AD-Certificate
[X] Certification Authority                 ADCS-Cert-Authority
```



Sysadmin Note — Why Only the Certification Authority Role Service?

The AD CS role includes several additional services: Web Enrollment (for browser-based enrollment), NDES (for network device enrollment), Online Responder (for OCSP), and Certificate Enrollment Policy/Service web services. For a foundational lab demonstrating AD-integrated certificate enrollment via MMC or Group Policy, only the Certification Authority role service is required. Installing unnecessary role services increases the attack surface without adding value to this project's objectives.

02

Configure the Enterprise Root CA

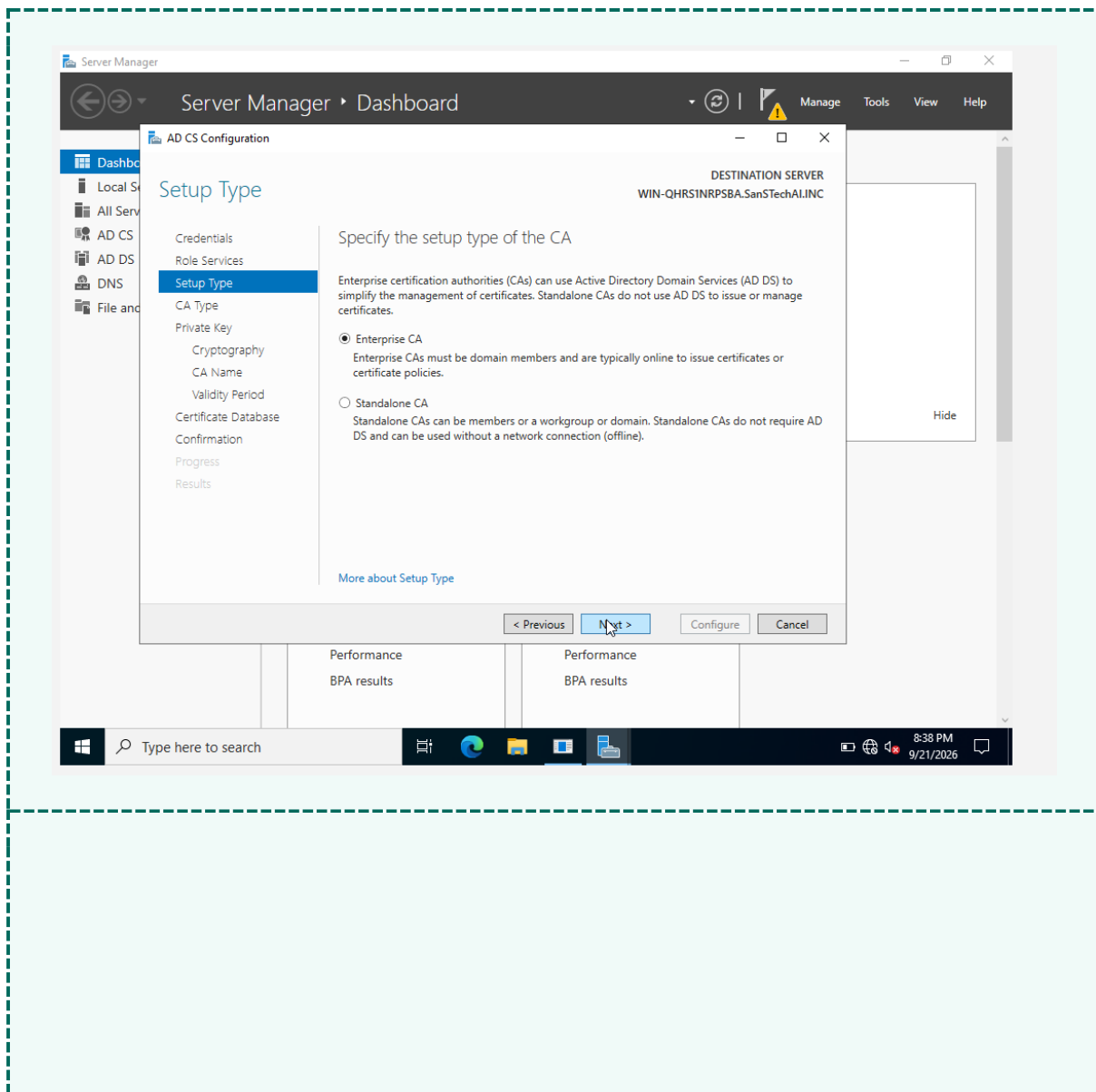
AD CS Configuration Wizard

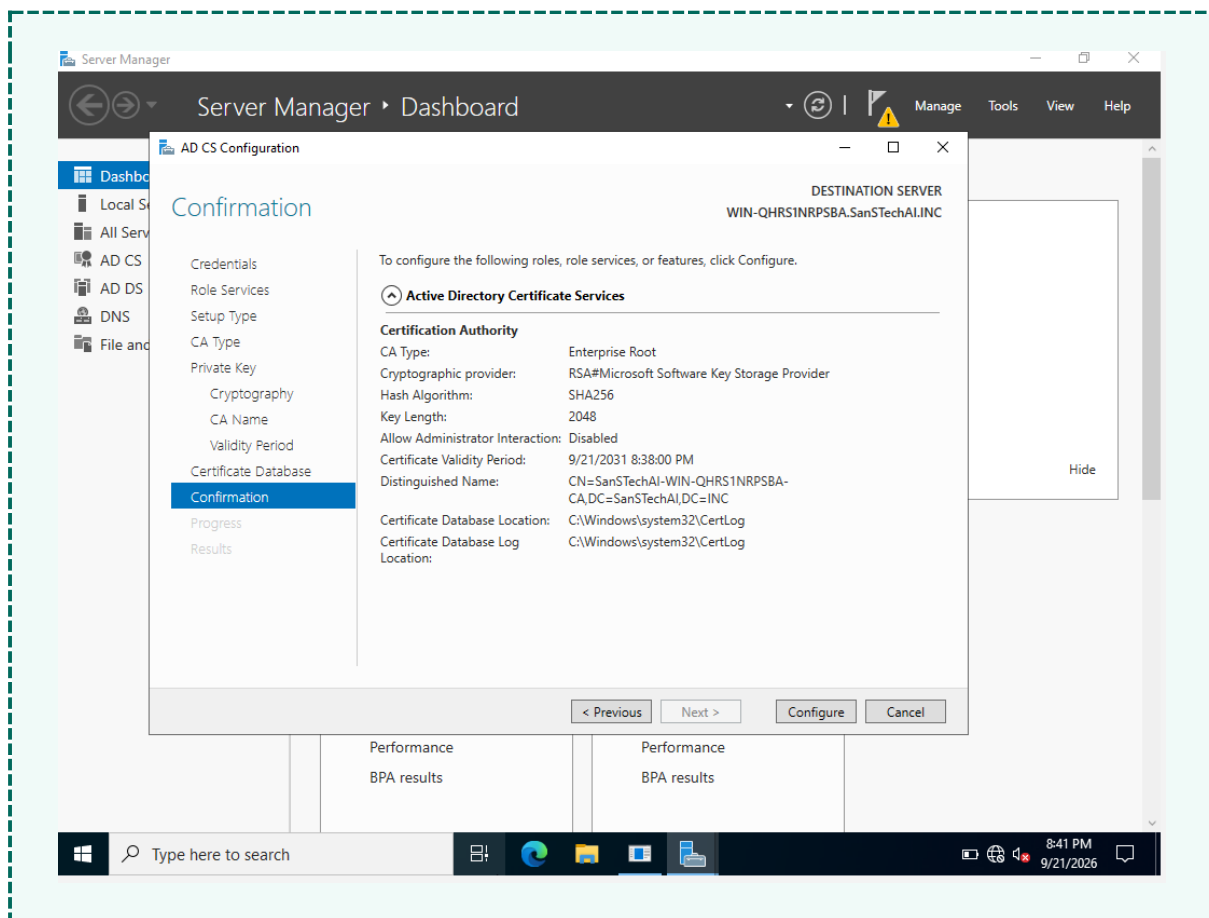
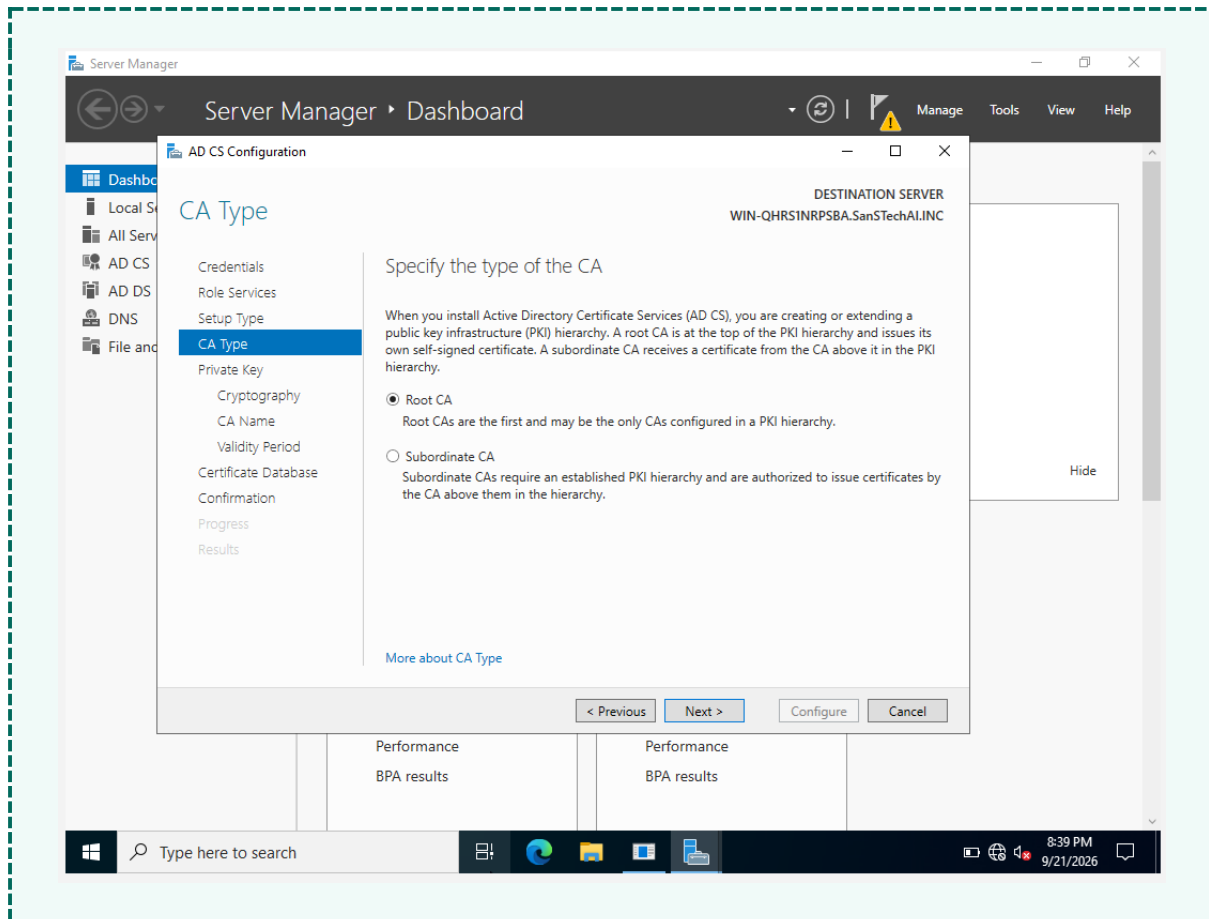
5.2 Step 2 — Configure the Enterprise Root CA

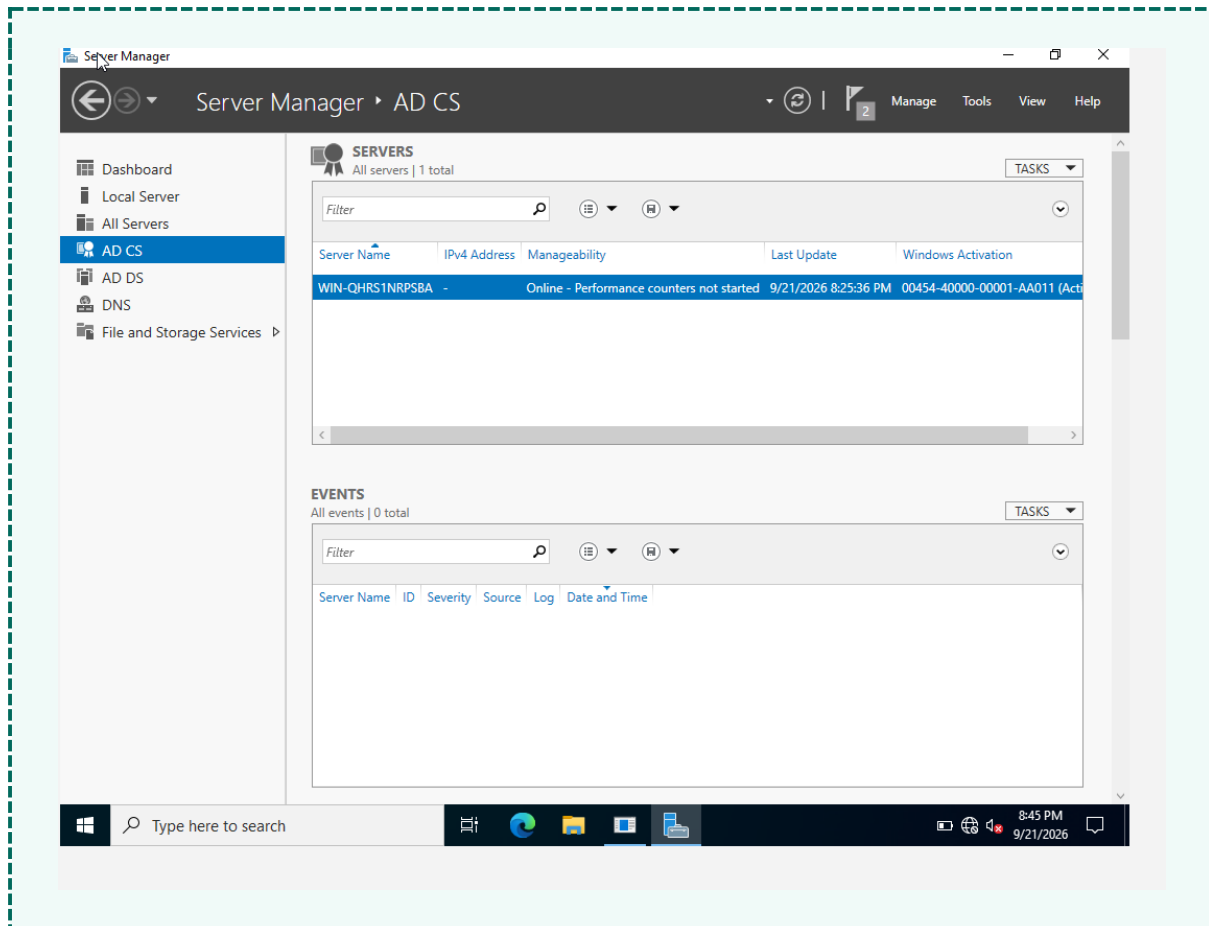
After the AD CS role was installed, the post-installation configuration wizard was launched to configure the Certificate Authority. An Enterprise Root CA was selected — this is the appropriate CA type for an Active Directory-integrated environment because it reads enrollment permissions from AD and automatically publishes certificates to the directory. A new CA private key was created.

Configuration Values Used

Configuration Setting	Value Selected / Rationale
Credentials	Administrator (local domain admin) — required to configure an Enterprise CA
Role Services to Configure	Certification Authority
Setup Type	Enterprise CA — integrates with Active Directory for enrollment policy and certificate publication
CA Type	Root CA — top-level CA in a new PKI hierarchy; no parent CA to sign this CA's certificate
Private Key	Create a new private key — generating a fresh, unique cryptographic identity for this CA
Cryptographic Provider	RSA#Microsoft Software Key Storage Provider
Key Length	2048 bits — standard for lab environments; production may use 4096 bits for root CAs
Hash Algorithm	SHA-256 — current standard; MD5 and SHA-1 are deprecated
CA Common Name	HEALTHCARE-ROOT-CA — reflects the domain/organisation context
Distinguished Name Suffix	DC=healthcare,DC=local
Validity Period	5 Years — appropriate for a Root CA certificate
Certificate Database	Default path: C:\Windows\system32\CertLog







Enterprise CA vs Standalone CA — Why Enterprise Was Chosen

An Enterprise CA is deeply integrated with Active Directory:

- Uses AD to store certificate templates, making them available for enrollment to domain users
- Automatically publishes issued certificates to the AD user object
- Supports automatic certificate enrollment through Group Policy
- Appears as a trusted CA automatically in domain-joined machines' certificate stores

A Standalone CA has none of these AD integrations — it is appropriate for workgroup environments or issuing certificates to non-domain entities. For this lab's purpose of demonstrating identity-linked certificate issuance via the healthcare.local domain, an Enterprise CA is the only appropriate choice.

03

Enable the EFS Certificate Template

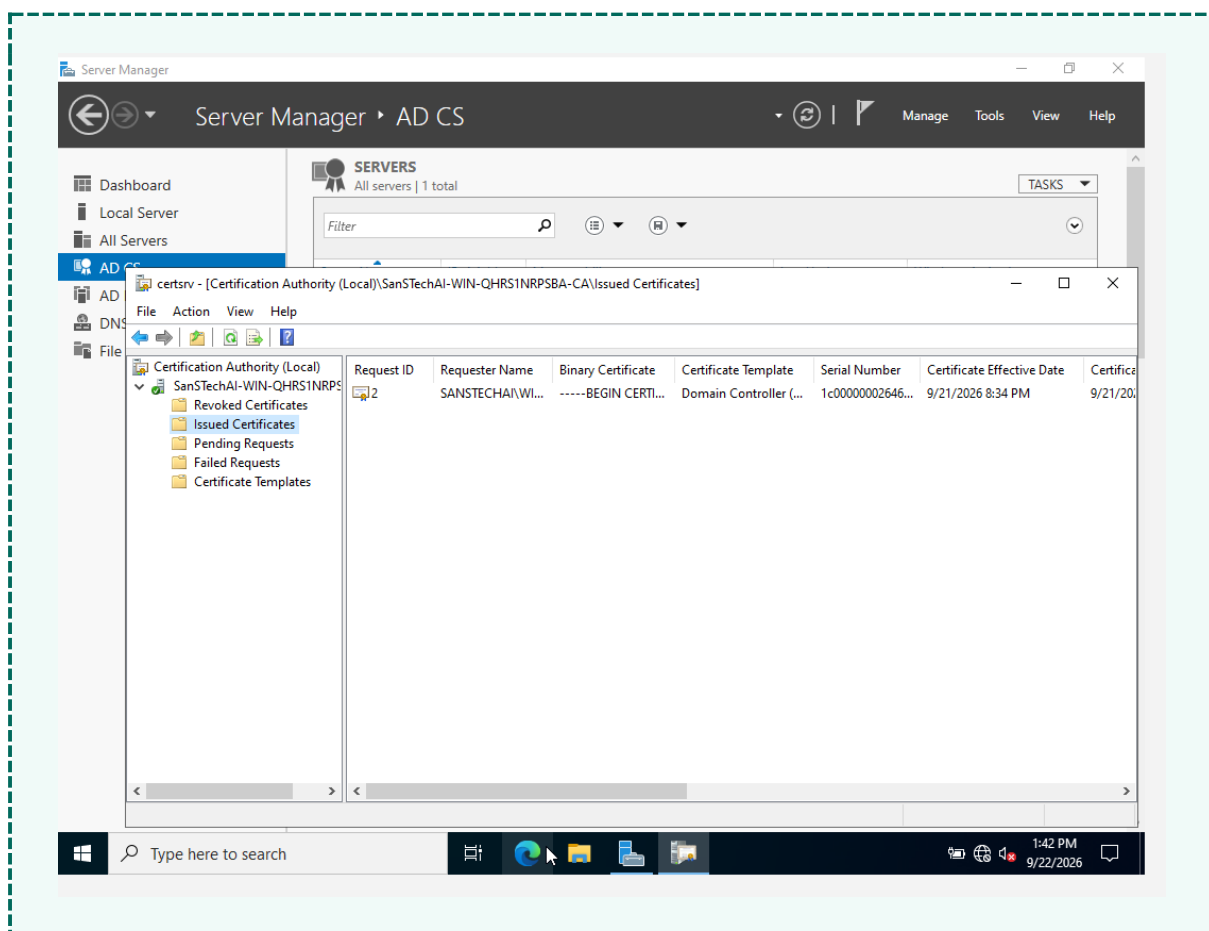
Certification Authority MMC
(certsrv.msc)

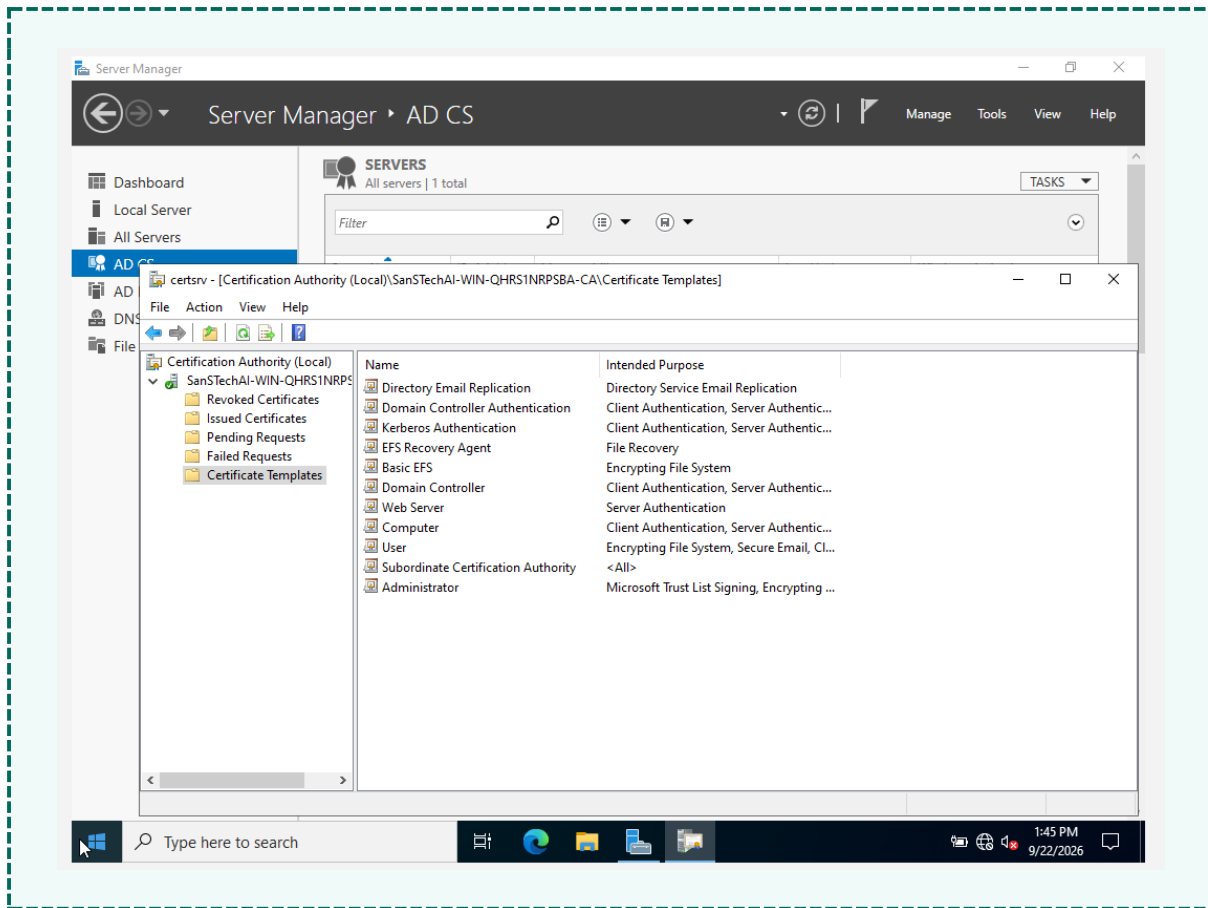
5.3 Step 3 — Enable the EFS Certificate Template

Certificate templates define what a certificate can be used for, who can enroll for it, how long it is valid, and what cryptographic properties it has. Windows Server includes several built-in templates. For this project, the Basic EFS template was enabled on HEALTHCARE-ROOT-CA — this template is specifically designed to issue certificates that can be used by EFS to encrypt and decrypt files.

Enabling the Template in Certification Authority MMC

1. On IT-Server, opened certsrv.msc (Start > Run > certsrv.msc)
2. Expanded HEALTHCARE-ROOT-CA in the left panel
3. Right-clicked Certificate Templates > New > Certificate Template to Issue
4. The Enable Certificate Templates dialog appeared — listing all available templates
5. Selected Basic EFS from the list
6. Clicked OK — the template is now available for enrollment by domain users
7. Verified the template appeared under Certificate Templates in the CA console





04

Create Domain Users — ali.test & other.test

Active Directory Users and Computers (ADUC)

5.4 Step 4 — Create Domain Users

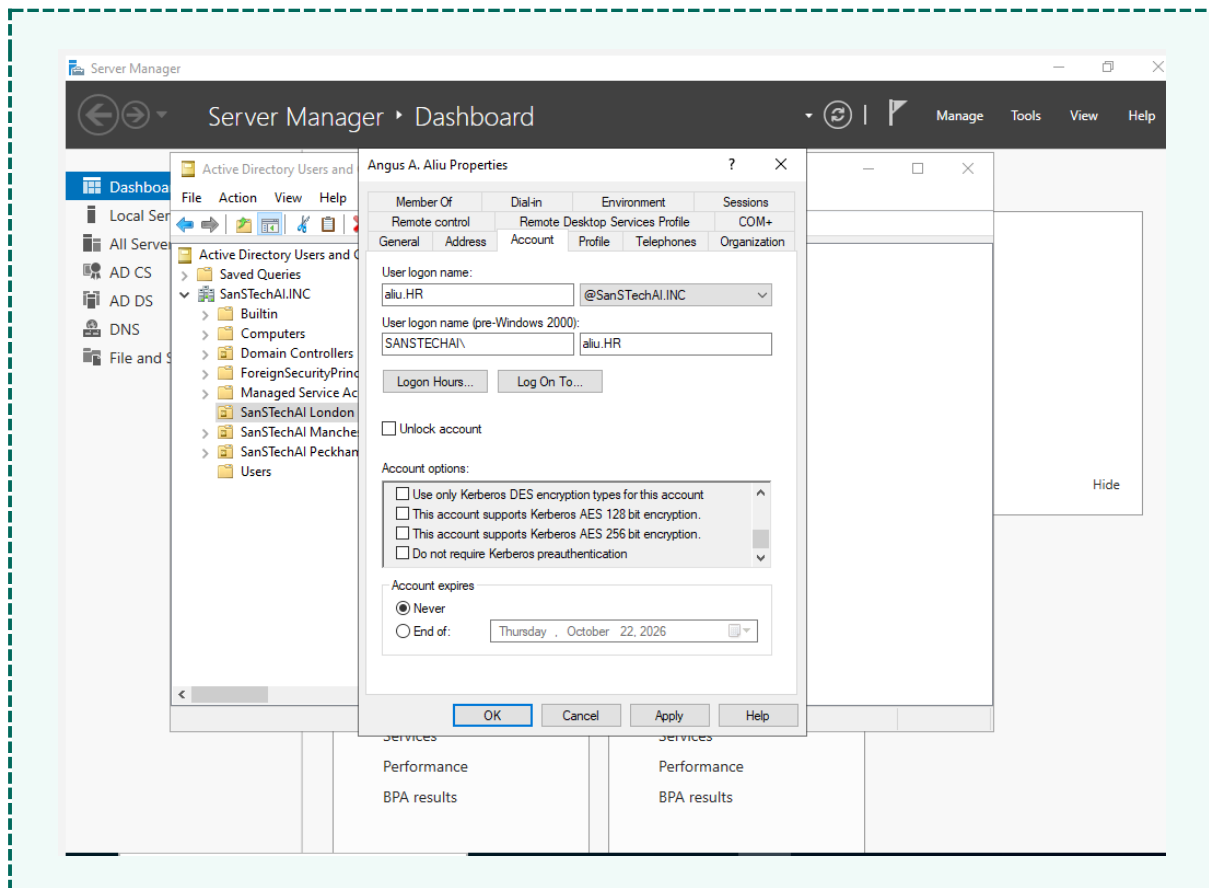
Two domain user accounts were created in Active Directory Users and Computers to serve as the test subjects for the EFS demonstration. ali.test is the certificate holder and file owner. other.test is the second user who will attempt — and fail — to decrypt ali.test's encrypted file.

Creating ali.test

8. On IT-Server, opened Active Directory Users and Computers (dsa.msc)
9. Navigated to healthcare.local > Users
10. Right-clicked Users > New > User
11. First name: ali | Last name: test | User logon name: ali.test
12. Set password and unchecked 'User must change password at next logon'
13. Completed the wizard — ali.test account created

Creating other.test

14. Repeated the same process: Right-click > New > User
15. First name: other | Last name: test | User logon name: other.test
16. Set password — same process as ali.test



05

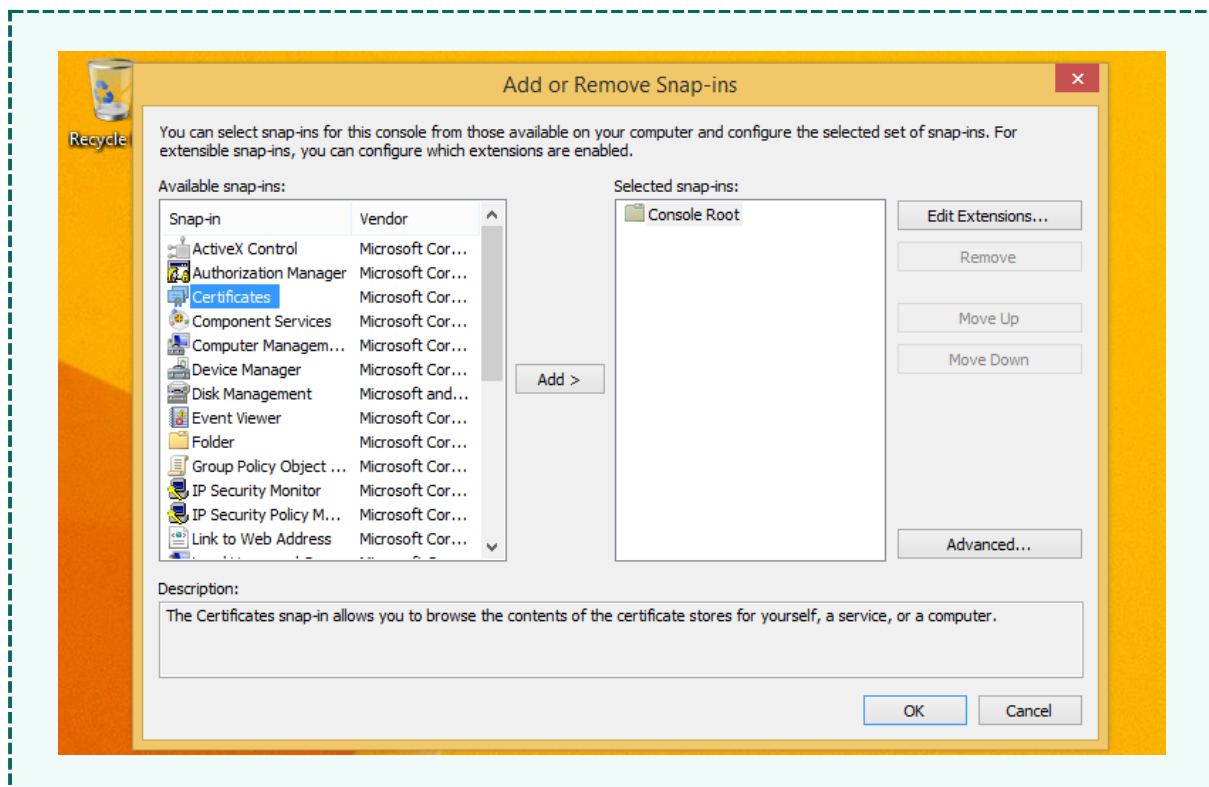
Enroll ali.test for an EFS Certificate

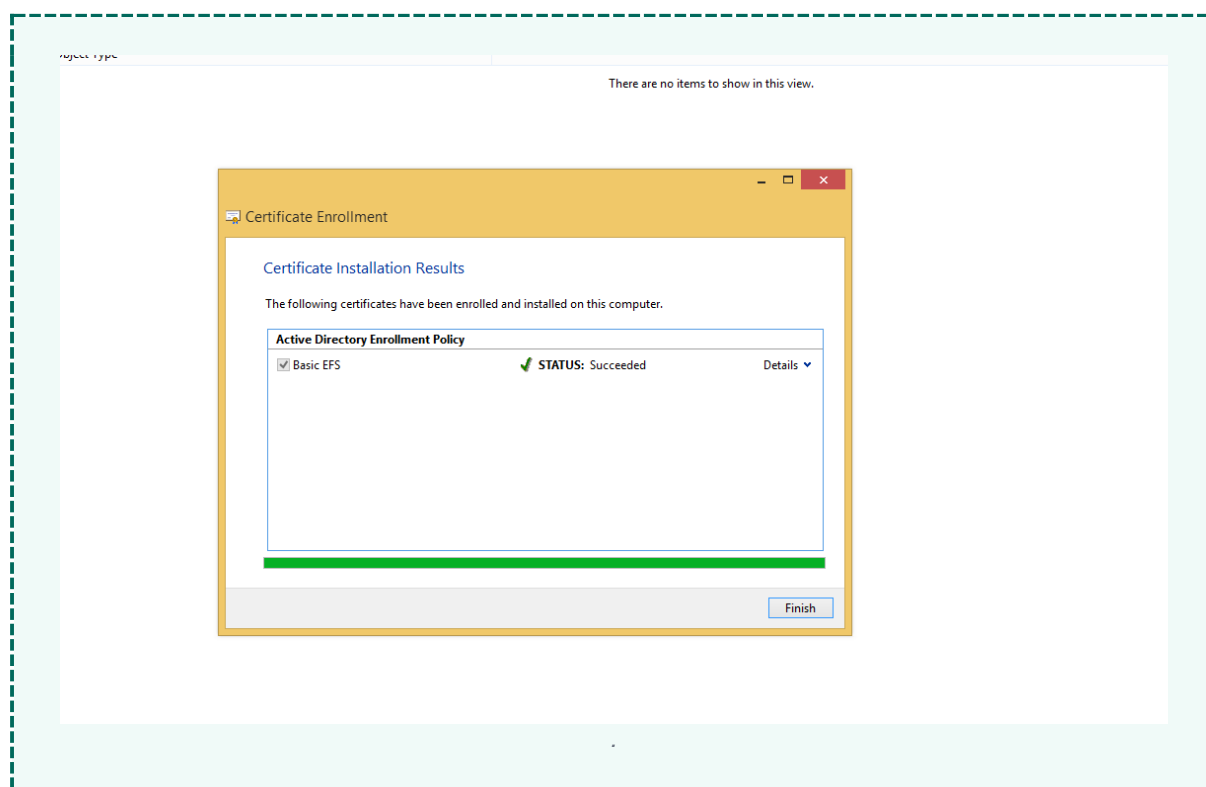
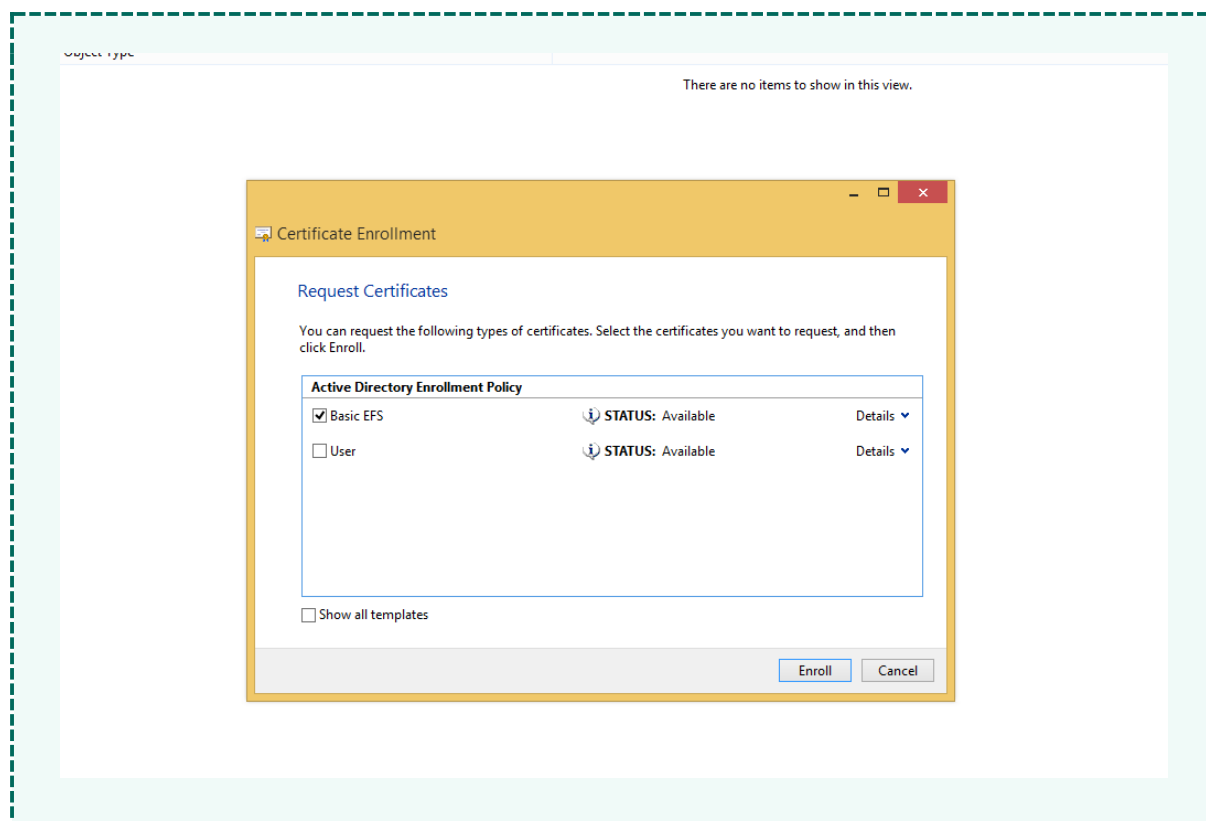
Certificates MMC (certmgr.msc) —
Windows 8 Client

5.5 Step 5 — Enroll ali.test for an EFS Certificate

With the CA configured and the EFS template enabled, ali.test was logged into the Windows 8 domain-joined client to request the EFS certificate. The Certificates MMC snap-in was used to submit a certificate enrollment request through the Active Directory Enrollment Policy — the mechanism by which domain users can request certificates from enterprise CAs.

17. On the Windows 8 client, logged in as ali.test (healthcare\ali.test)
18. Pressed Win+R, typed certmgr.msc, pressed Enter — opened the Current User Certificates MMC
19. In the left panel, navigated to Certificates — Current User > Personal
20. Right-clicked Personal > All Tasks > Request New Certificate
21. The Certificate Enrollment wizard launched
22. Select Certificate Enrollment Policy: Active Directory Enrollment Policy — clicked Next
23. The available certificate templates were displayed — selected Basic EFS
24. Clicked Enroll — the CA evaluated ali.test's identity against AD and issued the certificate
25. Status showed Succeeded — the EFS certificate was issued and stored in ali.test's Personal certificate store





06

Verify the Issued Certificate

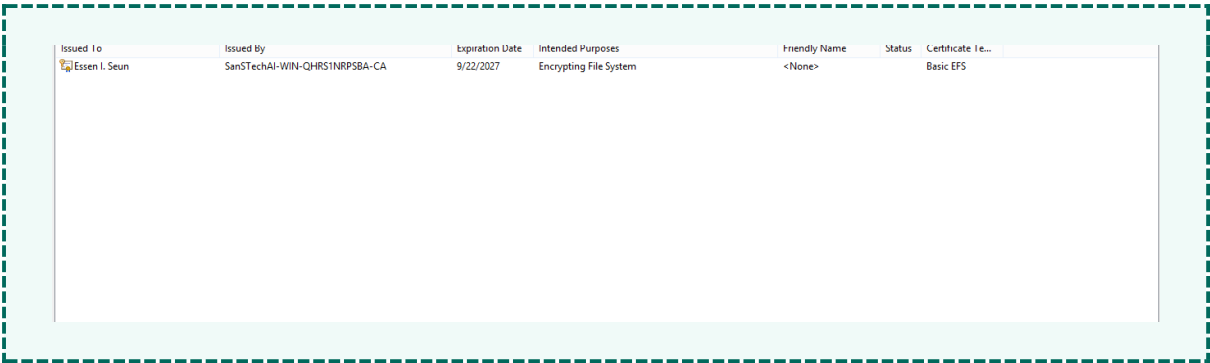
Certificates MMC — Current User
→ Personal → Certificates

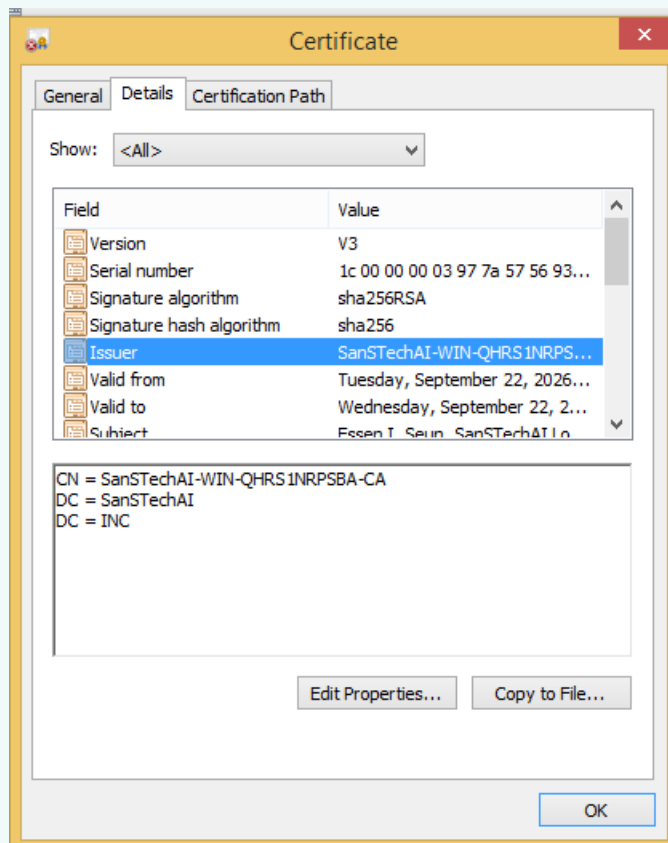
5.6 Step 6 — Verify the Issued Certificate

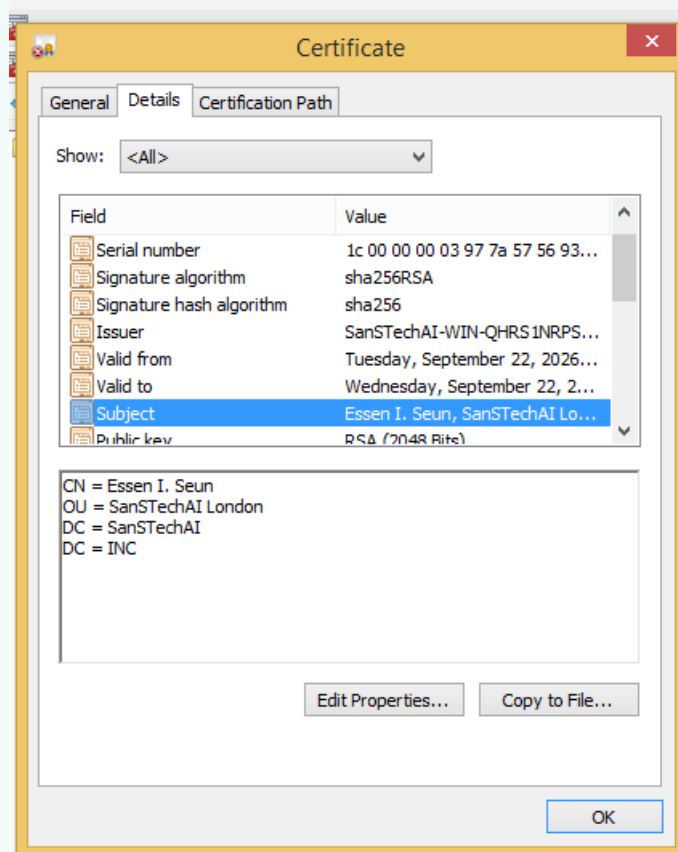
After successful enrollment, the issued certificate was inspected to confirm it was correctly issued by HEALTHCARE-ROOT-CA, contained ali.test's identity, and had the Encrypting File System purpose. This verification step is important — before using a certificate for encryption, a sysadmin should confirm the certificate details are correct to avoid encrypting files with an invalid or wrong-purpose certificate.

26. In certmgr.msc, navigated to Certificates — Current User > Personal > Certificates
27. The newly issued EFS certificate appeared in the right panel — issued to ali.test
28. Double-clicked the certificate to open the Certificate details dialog
29. Verified the following on the General tab:

Certificate Field	Expected / Confirmed Value
Issued to	ali.test (or ali.test@healthcare.local)
Issued by	HEALTHCARE-ROOT-CA
Valid from	[Issue date] — confirms the certificate is currently valid
Valid to	[Expiry date — typically 1-2 years from issuance for Basic EFS template]
Intended purposes	Encrypting File System — confirms this certificate is for EFS use
Public key	RSA (2048 bits) — matches the template's cryptographic requirements
Subject	CN=ali.test, DC=healthcare, DC=local
Issuer	CN=HEALTHCARE-ROOT-CA, DC=healthcare, DC=local







07

Create the Confidential Test File

Windows 8 Client — File Explorer /
Notepad

5.7 Step 7 — Create the Confidential Test File

A test directory and file were created to serve as the encryption subject. In a real healthcare environment, this file would represent a confidential patient record, financial data, or clinical research data that must be protected at the file-system level against unauthorised access — even by other staff with access to the same machine or file share.

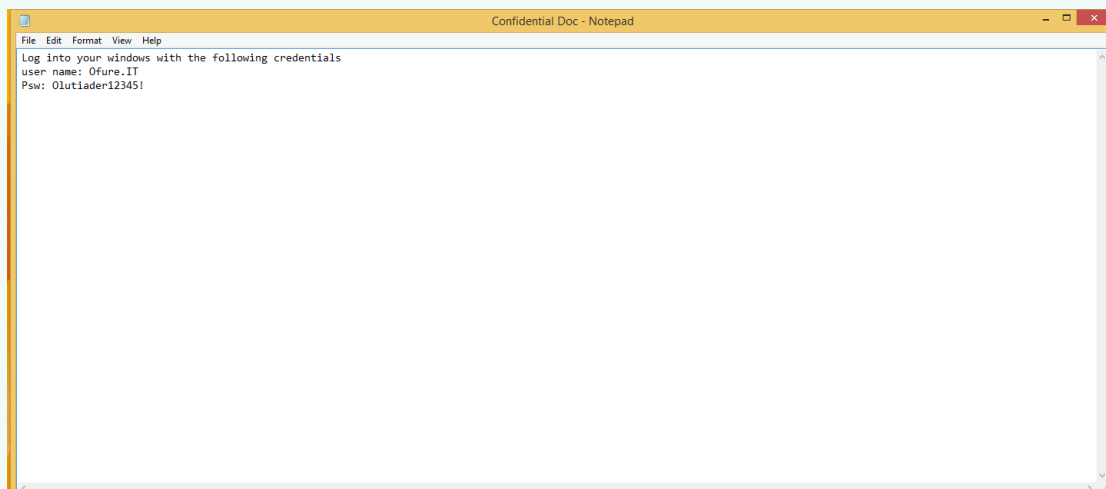
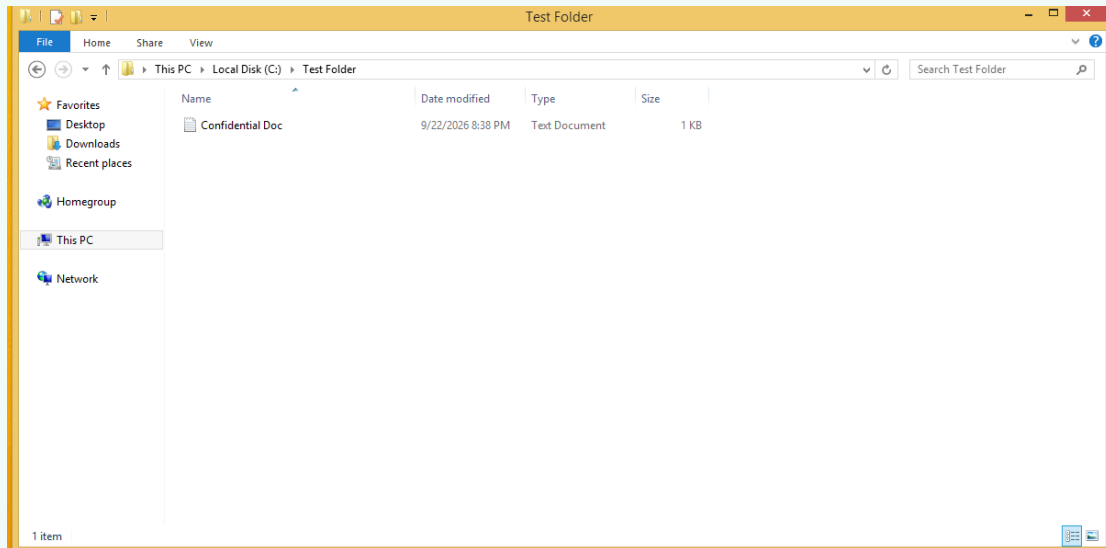
30. On the Windows 8 client (logged in as ali.test), opened File Explorer
31. Navigated to C:\ and created a new folder named EFS-Test
32. Opened Notepad and typed sample confidential content (e.g. patient name, date, notes)
33. Saved the file as C:\EFS-Test\confidential.txt
34. Closed Notepad

► Create the Test File — PowerShell Equivalent

```
# PowerShell equivalent - create test directory and file
New-Item -ItemType Directory -Path C:\EFS-Test
New-Item -ItemType File -Path C:\EFS-Test\confidential.txt

# Add sample content
Set-Content -Path C:\EFS-Test\confidential.txt -Value @"
HEALTHCARE - CONFIDENTIAL
Patient: [Sample Name]
DOB: [Date]
Diagnosis: [Sample Data]
Notes: This file is protected by EFS encryption.
Only ali.test can read this content.
"@

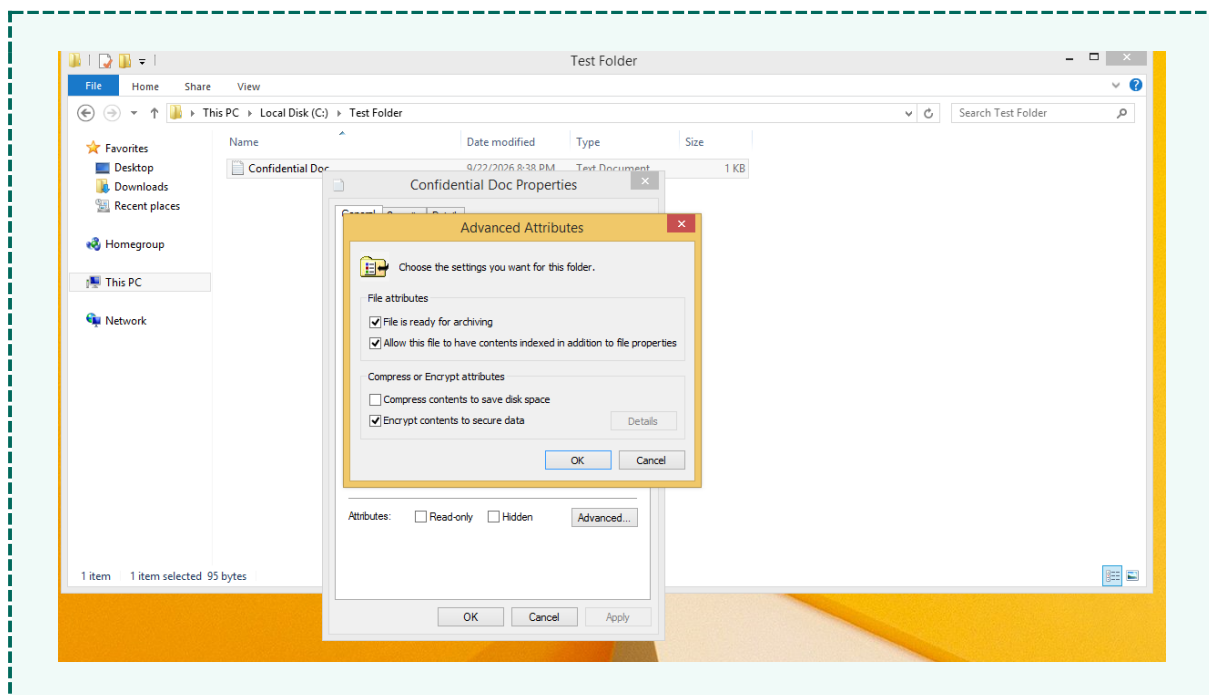
# Verify file was created
Get-Item C:\EFS-Test\confidential.txt
```

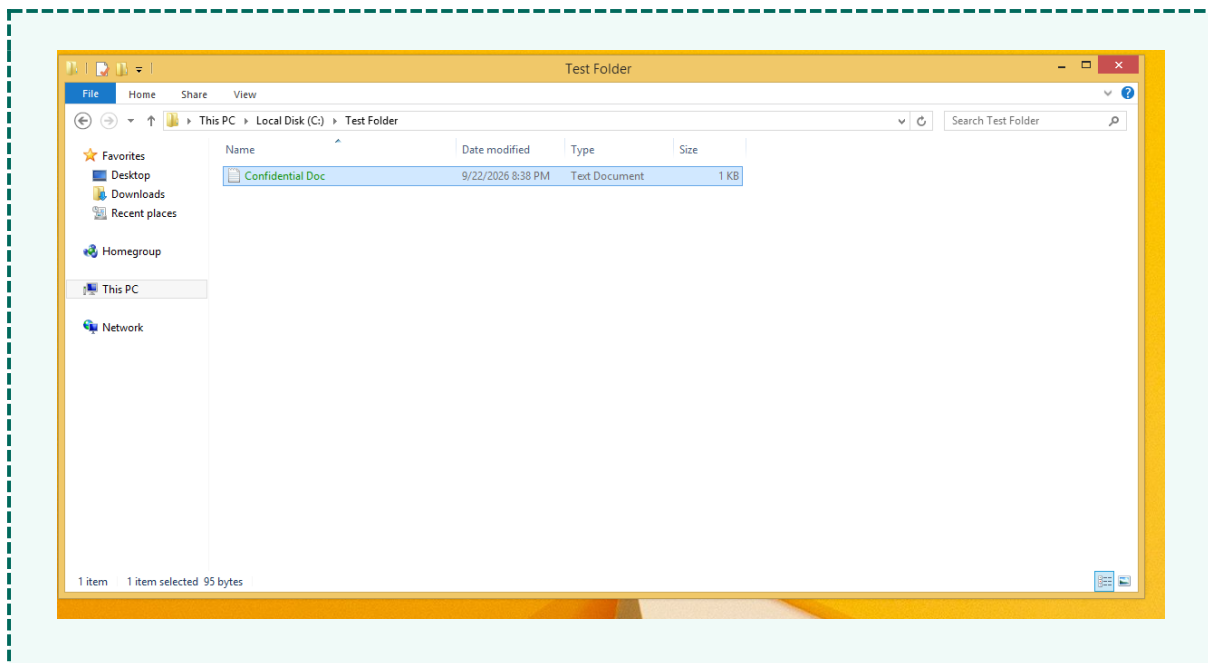


5.8 Step 8 — Encrypt the File with EFS

The test file was encrypted using EFS through the Windows file property dialog. This is the standard, GUI-driven method for applying EFS encryption. Under the hood, Windows generates a random File Encryption Key (FEK), encrypts the file content with the FEK using AES-256, then encrypts the FEK itself using ali.test's public key from the enrolled EFS certificate — ensuring only ali.test's private key can recover the FEK and thus the file.

35. Right-clicked C:\EFS-Test\confidential.txt in File Explorer > Properties
36. On the General tab, clicked Advanced
37. The Advanced Attributes dialog appeared
38. Checked the box: Encrypt contents to secure data
39. Clicked OK on the Advanced Attributes dialog
40. Clicked Apply on the file Properties dialog
41. Windows prompted: Apply changes to this folder, subfolders and files? — selected This file only
42. Clicked OK — EFS encryption was applied
43. The filename turned green in File Explorer — Windows' visual indicator that a file is EFS-encrypted





09

Verify Encryption with the cipher Command

Command Prompt — cipher /c

5.9 Step 9 — Verify Encryption with cipher Command

The cipher command is the built-in Windows command-line tool for inspecting and managing EFS encryption. The /c flag (certificate) displays the encryption certificate information for a specified file — including the user associated with the encryption and the certificate thumbprint. This command is the definitive way to verify that EFS encryption is active and confirm which certificate/key was used.

► cipher /c — EFS Encryption Verification

```
C:\> cipher /c C:\EFS-Test\confidential.txt

Listing C:\EFS-Test\
New files added to this directory will not be encrypted.

E confidential.txt
  Compatibility Level:
    Windows Vista/Server 2008

  Users who can decrypt:
    ali.test: Certificate thumbprint: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX
  Certificate Issuer: HEALTHCARE-ROOT-CA

  No recovery certificate found.

# Note: The 'E' prefix confirms the file is EFS Encrypted
# The certificate thumbprint matches the certificate issued in Step 5
# 'No recovery certificate found' = no EFS Data Recovery Agent configured
```

 [SCREENSHOT]**Command Prompt — cipher /c Output (Confirming EFS Encryption & Certificate)**

Capture the Command Prompt on the Windows 8 client (logged in as ali.test) showing the full output of 'cipher /c C:\EFS-Test\confidential.txt'. The output should clearly show: the 'E' prefix for the encrypted file, 'ali.test' under 'Users who can decrypt', and the certificate issuer 'HEALTHCARE-ROOT-CA'.

Understanding cipher /c Output

'E' prefix: Confirms the file is EFS encrypted (files with 'U' are unencrypted)

'Users who can decrypt': Lists every certificate/user that can open this file

Certificate thumbprint: The unique identifier of the EFS certificate used — matches the certificate issued to ali.test in Step 5

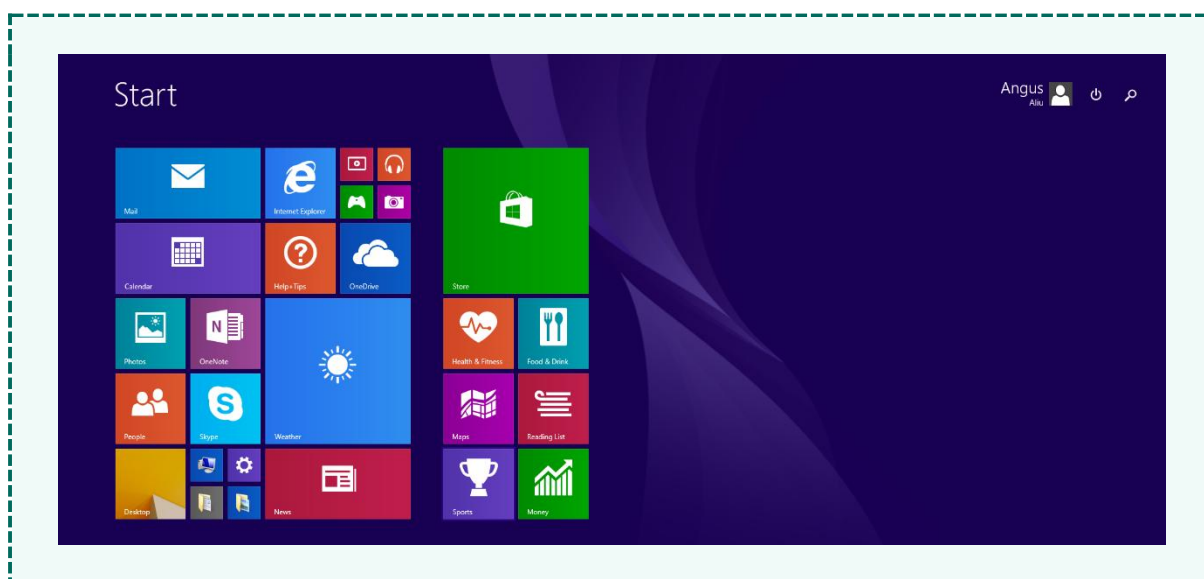
Certificate Issuer: HEALTHCARE-ROOT-CA — confirms the certificate came from our internal CA

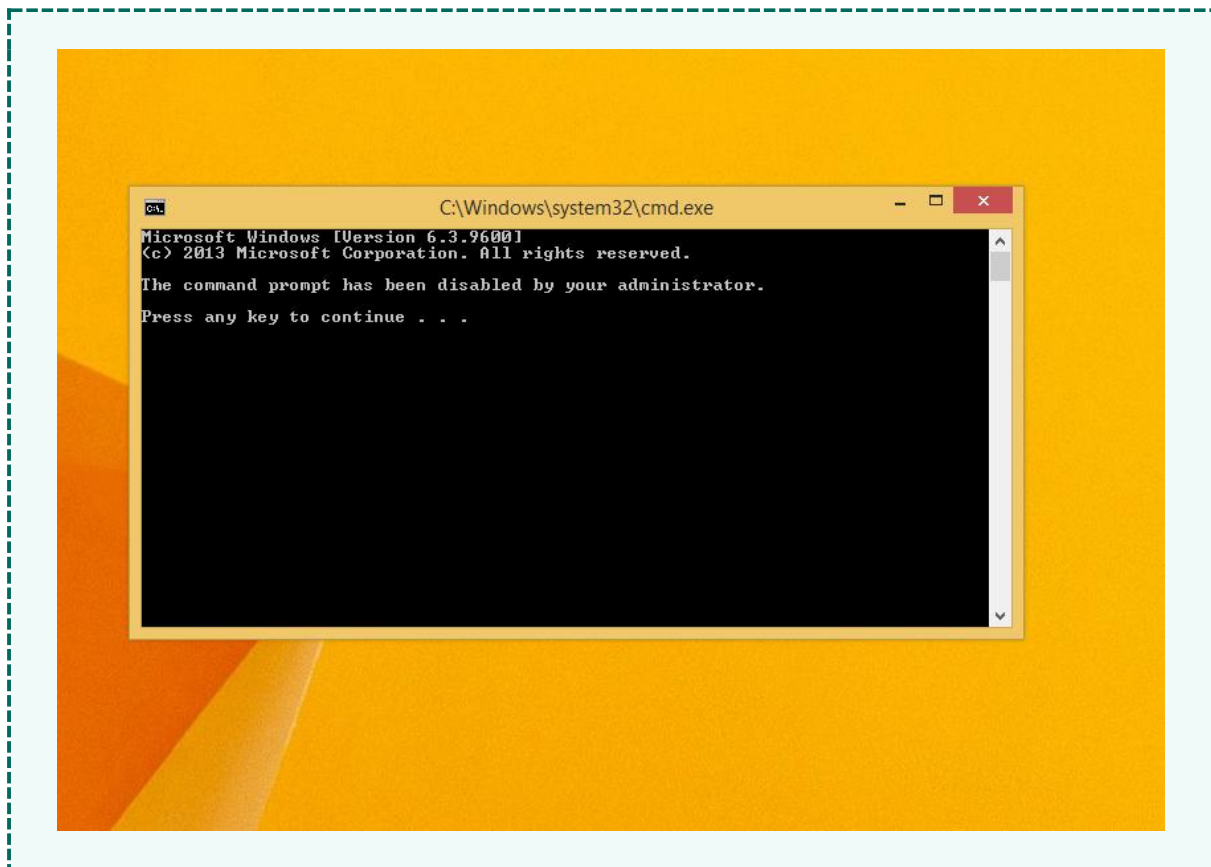
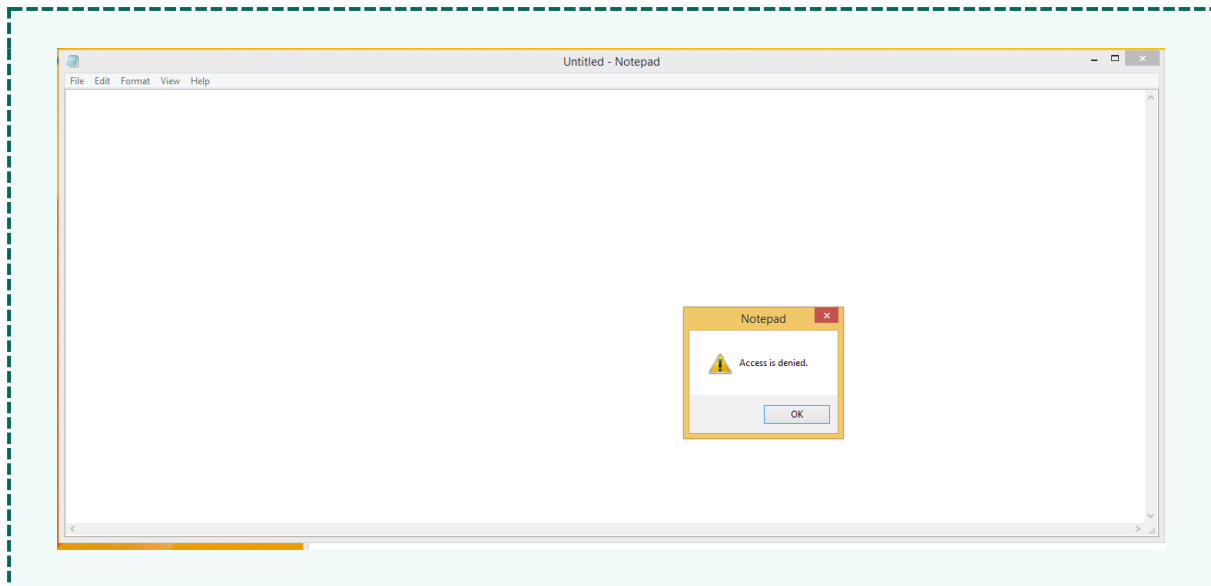
'No recovery certificate found': Indicates no EFS Data Recovery Agent (DRA) is configured — in production, a DRA should always be configured so that IT can recover encrypted files if a user leaves the organisation or loses their private key

5.10 Step 10 — Test Unauthorised Access (other.test)

The final and most important step demonstrates what EFS actually protects against. other.test is a fully valid domain user in healthcare.local — they can log into the domain, access network shares (subject to NTFS permissions), and use all standard domain resources. However, other.test has not enrolled for an EFS certificate and therefore does not have a private key that can decrypt the FEK protecting ali.test's confidential.txt. This step proves that domain account possession alone is insufficient to decrypt an EFS-protected file.

44. On the Windows 8 client, signed out of ali.test's session (or used Switch User)
45. Logged in as other.test (healthcare\other.test)
46. Opened File Explorer and navigated to C:\EFS-Test\
47. Double-clicked confidential.txt
48. Windows attempted to open the file in Notepad
49. Access was denied — Notepad displayed: Access is denied. You might not have permission to use this file. Contact the file's owner or an administrator to obtain permission.
50. Attempted to copy the file to another location — also denied
51. Ran cipher /c from Command Prompt to inspect the file — confirmed other.test has no entry in 'Users who can decrypt'





6. Technical Deep Dive — How EFS Encryption Works

This section documents the underlying cryptographic mechanics of the EFS encryption operation performed in Steps 8–10, for sysadmins and security engineers who need to understand the full technical picture rather than just the procedure.

Stage	Operation	Technical Detail
1	FEK Generation	When ali.test encrypts confidential.txt, Windows EFS generates a unique random File Encryption Key (FEK). This is a symmetric key — typically AES-256. Each file gets its own unique FEK.
2	File Encryption	The file data is encrypted using the FEK in AES-256 mode. The encrypted file content is written to the NTFS data stream of confidential.txt.
3	FEK Protection	The FEK itself must be protected. EFS retrieves ali.test's EFS certificate from the Personal store and uses the public key contained in that certificate to encrypt the FEK using RSA asymmetric encryption.
4	DDF Storage	The encrypted FEK (now called a Data Decryption Field entry, or DDF) is stored in the file's \$EFS alternate data stream — a hidden NTFS attribute on the file. The DDF contains the encrypted FEK plus the certificate thumbprint indicating which key was used.
5	Transparent Access (ali.test)	When ali.test opens the file: NTFS passes the request to the EFS driver. EFS reads the DDF from \$EFS. EFS uses ali.test's private key (from the user's protected key store) to decrypt the DDF and recover the FEK. EFS decrypts the file data using the FEK. The plaintext is handed to the application (Notepad). All of this happens transparently — ali.test just double-clicks and the file opens.
6	Denial (other.test)	When other.test opens the file: EFS reads the DDF. EFS finds only an entry for ali.test's certificate thumbprint. EFS looks for a private key corresponding to ali.test's certificate in other.test's key store — and finds none. EFS returns ACCESS_DENIED. The FEK cannot be recovered. The file cannot be decrypted.

7. Verification & Test Results

Test	Action Performed	Expected Result	Actual Result	Security Meaning
T-1	ali.test opens C:\EFS-Test\confidential.txt	File opens successfully in Notepad — full content readable	PASS ✓	ali.test holds the private key matching the EFS certificate — decryption is transparent
T-2	other.test opens C:\EFS-Test\confidential.txt	Access Denied — file cannot be opened	PASS ✓	Cryptographic access control enforced — domain account alone is insufficient
T-3	other.test copies the file to a personal folder and tries to open	Copy may succeed but open still fails — content is encrypted in the copy	PASS ✓	EFS encryption travels with the file — copying to another NTFS location retains encryption
T-4	cipher /c confirms ali.test certificate under 'can decrypt'	Certificate issuer = HEALTHCARE-ROOT-CA; ali.test listed	PASS ✓	Confirmed the certificate came from the internal CA and is bound to ali.test's identity
T-5	Certificate issuer confirmed as HEALTHCARE-ROOT-CA	Issued by: CN=HEALTHCARE-ROOT-CA,DC=healthcare,DC=local	PASS ✓	The trust chain is anchored in the internal CA — not a self-signed or third-party cert
T-6	cipher /c from other.test shows no entry for other.test	Only ali.test appears in the 'Users who can decrypt' list	PASS ✓	Proves the DDF contains only ali.test's certificate reference — no other user has access

All 6 Tests Passed — Project Objectives Met

Every test case produced the expected result. The lab demonstrates a complete, functioning PKI chain: AD DS identity → AD CS certificate issuance → EFS file encryption → private-key-enforced access control. The project provides direct, reproducible evidence of cryptographic access control operating within an Active Directory environment.

8. Security Concepts Demonstrated

Security Concept	How It Appears in This Lab	Real-World Relevance
Identity Management	Active Directory provides the domain identities (ali.test, other.test) that the entire system is built on. Without a valid AD identity, certificate enrollment is not possible	AD-based identity is the foundation of enterprise authentication — this lab shows how it extends into cryptographic operations, not just login
Public Key Infrastructure	AD CS establishes HEALTHCARE-ROOT-CA as the internal trusted root. All EFS certificates chain back to this CA — creating a verifiable, organisation-controlled trust anchor	Organisations with an internal PKI can issue certificates for many purposes: EFS, S/MIME email signing, code signing, VPN/Wi-Fi authentication, smart card login
Public-Key Cryptography	The EFS certificate contains ali.test's public key. The corresponding private key is held in ali.test's protected key store. The public key encrypts the FEK; the private key decrypts it	The RSA/asymmetric model — encrypt with public key, decrypt with private key — underpins TLS, S/MIME, SSH, and most modern secure communications
Data Confidentiality	The EFS-encrypted file cannot be read by other.test even if they can see the file in File Explorer. The content is mathematically protected by AES-256 encryption	Healthcare organisations processing patient data must protect records at rest. EFS provides file-level encryption that can complement BitLocker (disk-level) and TLS (transport-level) encryption
Access Control	Possession of a valid domain account (other.test) does not grant the cryptographic capability to read ali.test's encrypted file. Access is controlled by private key possession, not user account status	This is the principle of defence-in-depth in action: even if NTFS permissions were misconfigured and other.test could 'see' the file, EFS means they still cannot read the content
Key & Certificate Lifecycle	The cipher /c output shows the certificate thumbprint — this thumbprint changes if the certificate is renewed or re-issued. Private key loss = data loss unless a DRA or key archive is in place	Certificate lifecycle management is critical in production: organisations must track expiry dates, revocation, and ensure private key backup/recovery procedures exist before encrypting production data

9. Framework Alignment — ISO 27001 & NIST CSF

9.1 ISO/IEC 27001:2022 Control Alignment

Control	Control Name	How This Lab Satisfies / Demonstrates the Control
A.5.15	Access Control	EFS demonstrates cryptographic access control operating independently of NTFS permissions — showing that data confidentiality can be enforced at the file level through possession of a private key, not solely through access control list membership
A.5.16	Identity Management	AD DS provides the identity foundation. Ali.test's certificate is issued based on their AD identity — demonstrating that identity management and cryptographic operations are integrated through the PKI
A.5.18	Access Rights	The test demonstrates that domain account possession (other.test) does not automatically grant access rights to encrypted files. Access rights are derived from cryptographic capability (private key), not just user provisioning
A.8.2	Privileged Access Rights	The CA was configured using domain admin credentials. Admin rights are required for CA installation, template management, and certificate issuance policy — demonstrating that privileged access is required to manage PKI infrastructure
A.8.24	Use of Cryptography	The project directly demonstrates appropriate use of cryptography: RSA for asymmetric key protection (FEK encryption), AES-256 for symmetric file content encryption, SHA-256 for certificate hashing — all current standards for data protection at rest

9.2 NIST Cybersecurity Framework (CSF) Alignment

CSF Function	Category	How This Lab Contributes
PROTECT	Identity Management (PR.AA)	AD DS manages user identities and AD CS binds those identities to cryptographic certificates — directly supporting identity-based access control
PROTECT	Access Control (PR.AA-05)	EFS demonstrates file-level cryptographic access control — only the certificate holder can decrypt the file, regardless of other access control settings
PROTECT	Data Security (PR.DS-01)	EFS encrypts data at rest at the file level using AES-256 — directly implementing the data-at-rest protection requirement for confidential healthcare information
PROTECT	Protective Technology (PR.PS)	AD CS is a protective technology providing a PKI infrastructure that enables encryption, authentication, and digital signing across the organisation

10. Key Lessons & Sysadmin Notes

#	Lesson	Detail & Sysadmin Implication
L-1	AD CS is a PKI service — it is not the same as encryption	AD CS issues the certificates that enable encryption. It does not encrypt anything itself. A common misunderstanding is conflating the CA with the encryption mechanism. In this lab: AD CS issued the certificate; EFS performed the encryption using the certificate's public key.
L-2	A certificate contains a public key — the private key is the critical asset	The certificate (which includes the public key) can be shared freely and published in AD. The private key, stored in ali.test's user profile, is the capability that enables decryption. Private key protection and backup are the operational priority in any PKI deployment.
L-3	NTFS permissions and EFS are independent security layers	An administrator with 'Full Control' NTFS permission on confidential.txt cannot read its contents without the EFS private key. Conversely, granting NTFS read permission to other.test would allow them to see the file in File Explorer but not read its content. Both layers should be configured correctly in production.
L-4	Private key loss means data loss without a recovery mechanism	If ali.test's profile is deleted, the machine reimaged, or the certificate revoked before the file is decrypted, the data is permanently inaccessible. This is why the cipher /c output showed 'No recovery certificate found' — in production, an EFS Data Recovery Agent (DRA) must be configured before deploying EFS to avoid data loss scenarios.
L-5	The Enterprise CA integrates with AD for automatic trust distribution	When HEALTHCARE-ROOT-CA was configured as an Enterprise CA, its certificate was automatically published to all domain-joined machines' Trusted Root Certification Authorities store via Group Policy. This means ali.test's EFS certificate is automatically trusted by every machine in healthcare.local — no manual certificate import needed.
L-6	EFS encryption travels with the file on NTFS volumes	Copying an EFS-encrypted file to another NTFS location retains the encryption. Copying to a FAT32 or exFAT drive strips the encryption (because those file systems cannot support NTFS alternate data streams where the DDF is stored) — this is a data loss risk and data leak risk that must be addressed in DLP policy.

11. Recommended Next Phases

This project establishes the foundation. The following extensions build on this PKI infrastructure to demonstrate progressively more sophisticated enterprise capabilities.

Phase	Next Project	What It Adds
2	AD CS Certificate Auto-Enrollment via Group Policy	Configure a GPO to automatically enroll all domain users for EFS certificates at login — demonstrating enterprise-scale certificate provisioning without manual user action. This is how production environments manage certificates at scale.
3	EFS Data Recovery Agent (DRA) Configuration	Configure an EFS Data Recovery Agent certificate and policy so that a designated admin can decrypt any EFS-protected file in the domain. This addresses the data loss risk identified in Lesson L-4 and is required before EFS can be safely deployed in production.
4	Certificate Revocation & CRL Distribution	Revoke ali.test's EFS certificate and observe that the file becomes inaccessible (after the certificate revocation is propagated). Deploy a CRL Distribution Point (CDP) to demonstrate how revocation information is distributed to clients.
5	Smart Card / Certificate-Based Domain Logon	Issue smart card logon certificates to domain users — demonstrating that AD CS can serve as the PKI backend for certificate-based authentication (replacing password-based login), directly supporting the strongest form of identity assurance.
6	S/MIME Email Signing and Encryption	Issue S/MIME certificates and configure Outlook to sign and encrypt email messages using certificates issued by HEALTHCARE-ROOT-CA — demonstrating another major use case for the internal PKI built in this project.

12. Screenshot Evidence Index

The following table indexes all screenshot placeholders in this document. Capture each screenshot during the live build and insert it at the corresponding location.

#	Screenshot Caption	Section	Source
1	VirtualBox — Both VMs Running (IT-Server and Windows 8 Client)	3.1 — Lab Environment	VirtualBox Manager
2	Server Manager — AD CS Role Selected (Installation Step)	5.1 — Step 1	Server Manager Wizard
3	Server Manager — AD CS Installation Complete (Post-Install Notification)	5.1 — Step 1	Server Manager
4	AD CS Configuration Wizard — Enterprise Root CA Type Selection	5.2 — Step 2	AD CS Config Wizard
5	AD CS Configuration Wizard — CA Name (HEALTHCARE-ROOT-CA)	5.2 — Step 2	AD CS Config Wizard
6	Certification Authority MMC — HEALTHCARE-ROOT-CA Running	5.2 — Step 2	certsrv.msc
7	Certification Authority MMC — Enable Certificate Template (Basic EFS)	5.3 — Step 3	certsrv.msc
8	Certification Authority MMC — Certificate Templates List (Basic EFS)	5.3 — Step 3	certsrv.msc
9	Active Directory Users and Computers — ali.test & other.test Created	5.4 — Step 4	dsa.msc
10	Certificates MMC — Request New Certificate (AD Enrollment Policy)	5.5 — Step 5	certmgr.msc (Win8)
11	Certificates MMC — Basic EFS Template Selected for Enrollment	5.5 — Step 5	certmgr.msc (Win8)
12	Certificates MMC — Enrollment Status: Succeeded	5.5 — Step 5	certmgr.msc (Win8)
13	Certificate Details — ali.test EFS Certificate (General Tab)	5.6 — Step 6	certmgr.msc (Win8)
14	Certificate Details — Details Tab (Subject & Issuer)	5.6 — Step 6	certmgr.msc (Win8)
15	Windows 8 — confidential.txt Created in C:\EFS-Test (Pre-Encryption)	5.7 — Step 7	File Explorer / Notepad
16	File Properties → Advanced — 'Encrypt contents' Checkbox Ticked	5.8 — Step 8	File Properties Dialog
17	File Explorer — confidential.txt Green Filename (EFS Encrypted)	5.8 — Step 8	File Explorer (Win8)
18	Command Prompt — cipher /c Output (ali.test Can Decrypt, HEALTHCARE-ROOT-CA)	5.9 — Step 9	CMD (Win8)

19	Windows 8 — other.test Logged In (Domain Login Confirmed)	5.10 — Step 10	Win8 Desktop
20	Windows 8 — Access Denied: other.test Cannot Open confidential.txt	5.10 — Step 10	Error Dialog (Win8)
21	Command Prompt — cipher /c as other.test (Only ali.test Listed)	5.10 — Step 10	CMD (Win8 as other.test)

End of Build Documentation

AD CS → EFS Project | HEALTHCARE-ROOT-CA | healthcare.local | Windows Server 2022 | Windows 8